Aula 03 — Operadores e expressões

Cálculos, comparações e lógica para preparar a tomada de decisão

Dieimes Nunes

Retomando as Aulas 01 e 02

Na **Aula 01**, vimos que programar começa antes do código:

- entender o problema;
- organizar os passos;
- identificar entrada, processamento e saída;
- representar a solução em algoritmo e pseudocódigo;
- criar os primeiros programas com `print()` e `input()`.

Na **Aula 02**, avançamos para:

- variáveis;
- tipos de dados;
- atribuição;
- entrada de dados;
- conversão com `int()` e `float()`;
- primeiros cálculos em Python.

Hoje vamos aprofundar os **operadores** e as **expressões**.

Uma situação para começar

Imagine um sistema simples de uma loja.

O usuário informa:

- nome do produto;
- preço unitário;
- quantidade comprada;
- percentual de desconto.

O programa precisa calcular:

- subtotal;
- valor do desconto;
- total a pagar.

Antes de criar o código, precisamos montar as expressões matemáticas corretamente.

Pensando antes do código

Se um produto custa **R$ 50,00** e a pessoa compra **3 unidades**, temos:

```
subtotal = preço × quantidade
subtotal = 50 × 3
subtotal = 150
```

Se o desconto é de **10%**, temos:

```
valor_desconto = subtotal × 10 / 100
valor_desconto = 150 × 10 / 100
valor_desconto = 15
```

Então:

```
total = subtotal - valor_desconto
total = 150 - 15
total = 135
```

Atividade 1 — Calculando no caderno

Resolva no caderno antes de usar o computador.

Uma loja vendeu:

- produto: teclado;
- preço unitário: R$ 80,00;
- quantidade: 2;
- desconto: 5%.

Calcule:

1. subtotal;
2. valor do desconto;
3. total a pagar.

Depois, compare o raciocínio com um colega.

Operadores aritméticos

Em Python, usamos operadores para realizar cálculos.

Operador	Significado	Exemplo
`+`	adição	`10 + 5`
`-`	subtração	`10 - 5`
`*`	multiplicação	`10 * 5`
`/`	divisão	`10 / 5`
`//`	divisão inteira	`10 // 3`
`%`	resto da divisão	`10 % 3`
`**`	potência	`2 ** 3`

Esses operadores aparecem em praticamente todos os programas.

Exemplos no Python

```python
print(10 + 5)
print(10 - 5)
print(10 * 5)
print(10 / 5)
print(10 // 3)
print(10 % 3)
print(2 ** 3)
```

Resultado esperado:

```
15
5
50
2.0
3
1
8
```

Observe que a divisão com `/` gera um resultado do tipo `float`.

Divisão comum e divisão inteira

Veja a diferença:

```
print(10 / 3)
print(10 // 3)
print(10 % 3)
```

Resultado:

```
3.3333333333333335
3
1
```

Interpretação:

- `10 / 3` mostra a divisão completa;
- `10 // 3` mostra apenas a parte inteira;
- `10 % 3` mostra o resto.

Onde o resto da divisão é útil?

O operador `%` é muito importante.

Ele pode ser usado para descobrir, por exemplo, se um número é par ou ímpar.

```
numero = 8
resto = numero % 2

print(resto)
```

Se o resto da divisão por 2 for `0`, o número é par.

Se o resto for `1`, o número é ímpar.

Ainda não vamos usar `if` profundamente, mas essa ideia será muito importante para a próxima etapa do curso.

Atividade 2 — Testando operadores

Crie um arquivo chamado:

`aula03_operadores.py`

Digite e execute:

```
print(20 + 4)
print(20 - 4)
print(20 * 4)
print(20 / 4)
print(20 // 3)
print(20 % 3)
print(2 ** 4)
```

Depois responda no caderno:

1. Qual operação gerou resultado decimal?
2. Qual operação mostrou o resto da divisão?
3. Qual operação representa potência?

Expressões aritméticas

Uma expressão aritmética combina valores, variáveis e operadores.

Exemplo:

```
nota1 = 7
nota2 = 8
nota3 = 9

media = (nota1 + nota2 + nota3) / 3

print(media)
```

A expressão principal é:

```
(nota1 + nota2 + nota3) / 3
```

Ela soma as notas e depois divide por 3.

Precedência de operadores

O Python segue uma ordem para resolver expressões.

Ordem simplificada:

1. parênteses;
2. potência;
3. multiplicação, divisão, divisão inteira e resto;
4. adição e subtração.

Exemplo:

```
resultado = 10 + 2 * 3
print(resultado)
```

Resultado:

```
16
```

Primeiro o Python calcula `2 * 3`, depois soma `10`.

Mudando a ordem com parênteses

```
resultado = (10 + 2) * 3
print(resultado)
```

Resultado:

```
36
```

Os parênteses deixam a intenção mais clara.

Na dúvida, use parênteses.

Um código claro é melhor do que um código difícil de entender.

Atividade 3 — Qual será o resultado?

Sem executar no Python, tente descobrir o resultado das expressões.

```
print(5 + 3 * 2)
print((5 + 3) * 2)
print(20 / 4 + 2)
print(20 / (4 + 1))
print(2 ** 3 + 1)
print(2 ** (3 + 1))
```

Depois execute no Python e confira.

Se errar, anote o motivo.

Correção da Atividade 3

Resultados esperados:

```
11
16
7.0
4.0
9
16
```

O ponto principal não é apenas acertar.

O importante é conseguir explicar a ordem de execução.

Pergunta para discussão:

> Em qual expressão os parênteses fizeram mais diferença?

Aplicando em um problema real

Agora vamos voltar ao problema da loja.

Precisamos receber:

- produto;
- preço;
- quantidade;
- desconto.

E calcular:

```
subtotal = preço × quantidade
valor_desconto = subtotal × desconto / 100
total = subtotal - valor_desconto
```

Esse é um exemplo de **estrutura sequencial**.

O programa executa uma instrução depois da outra, na ordem em que foi escrita.

Estrutura sequencial

A estrutura sequencial é a forma mais simples de organizar um programa.

Exemplo:

1. Ler o preço.
2. Ler a quantidade.
3. Calcular o subtotal.
4. Ler o desconto.
5. Calcular o valor do desconto.
6. Calcular o total.
7. Mostrar o resultado.

Não existe escolha ainda.

Não existe repetição ainda.

Apenas uma sequência de passos.

Programa: loja com desconto

```
produto = input("Digite o nome do produto: ")
preco = float(input("Digite o preço unitário: "))
quantidade = int(input("Digite a quantidade: "))
desconto = float(input("Digite o percentual de desconto: "))

subtotal = preco * quantidade
valor_desconto = subtotal * desconto / 100
total = subtotal - valor_desconto

print("----- RESUMO DA COMPRA -----")
print(f"Produto: {produto}")
print(f"Subtotal: R$ {subtotal:.2f}")
print(f"Desconto: R$ {valor_desconto:.2f}")
print(f"Total a pagar: R$ {total:.2f}")
```

Entendendo o programa

Nesta linha:

```
preco = float(input("Digite o preço unitário: "))
```

Acontecem duas coisas:

1. `input()` recebe o dado digitado pelo usuário;
2. `float()` converte o texto digitado para número real.

Sem a conversão, o Python trataria o valor como texto.

Isso atrapalharia os cálculos.

Atividade 4 — Loja com desconto

Crie um arquivo chamado:

```
loja_desconto.py
```

Implemente o programa da loja com desconto.

Depois teste com os dados:

```
Produto: Mouse
Preço: 50
Quantidade: 3
Desconto: 10
```

Resultado esperado:

```
Subtotal: R$ 150.00
Desconto: R$ 15.00
Total a pagar: R$ 135.00
```

Melhorando a saída

Podemos deixar a saída mais organizada usando linhas e textos explicativos.

```
print()
print("==============================")
print("        RESUMO DA COMPRA      ")
print("==============================")
print(f"Produto: {produto}")
print(f"Subtotal: R$ {subtotal:.2f}")
print(f"Desconto: R$ {valor_desconto:.2f}")
print(f"Total: R$ {total:.2f}")
```

A apresentação do resultado também faz parte da qualidade do programa.

Expressões relacionais

Além de calcular, programas também precisam comparar valores.

Operadores relacionais:

Operador	Significado	Exemplo
`>`	maior que	`idade > 18`
`<`	menor que	`idade < 18`
`>=`	maior ou igual	`nota >= 7`
`<=`	menor ou igual	`nota <= 10`
`==`	igual	`senha == "123"`

Operador	Significado	Exemplo
`!=`	diferente	`opcao != 0`

Essas expressões geram resultado `True` ou `False`.

Exemplos de comparações

```
idade = 17
nota = 8.5
senha = "1234"

print(idade >= 18)
print(nota >= 7)
print(senha == "1234")
print(senha != "1234")
```

Resultado:

```
False
True
True
False
```

Esses resultados são valores lógicos.

Cuidado com = e ==

Em Python:

```
idade = 18
```

significa atribuição.

Ou seja:

> guardar o valor 18 dentro da variável idade.

Já:

```
idade == 18
```

significa comparação.

Ou seja:

verificar se idade é igual a 18.

Atividade 5 — Comparações

Antes de executar, diga se cada expressão será `True` ou `False`.

```
print(10 > 5)
print(10 < 5)
print(7 >= 7)
print(8 <= 6)
print("Ana" == "Ana")
print("Ana" == "ana")
print("Python" != "Java")
```

Depois execute no Python e confira.

Atenção:

> Para textos, letras maiúsculas e minúsculas fazem diferença.

Expressões lógicas

Expressões lógicas combinam condições.

Em Python, usamos principalmente:

Operador	Significado
`and`	e
`or`	ou
`not`	negação

Esses operadores serão muito importantes quando estudarmos estruturas condicionais.

Operador `and`

O `and` exige que as duas condições sejam verdadeiras.

Exemplo:

```
idade = 20
tem_documento = True

print(idade >= 18 and tem_documento == True)
```

Resultado:

```
True
```

A pessoa tem 18 anos ou mais **e** possui documento.

Operador `or`

O `or` exige que pelo menos uma condição seja verdadeira.

Exemplo:

```
tem_cartao = False
tem_pix = True

print(tem_cartao == True or tem_pix == True)
```

Resultado:

```
True
```

A pessoa pode pagar com cartão **ou** pix.

Operador `not`

O `not` inverte o resultado lógico.

```
bloqueado = False

print(not bloqueado)
```

Resultado:

```
True
```

Se o usuário **não está bloqueado**, então ele pode continuar.

Atividade 6 — Expressões lógicas

Analise as variáveis:

```
idade = 16
possui_autorizacao = True
nota = 6.5
frequencia = 80
```

Escreva no caderno se as expressões serão `True` ou `False`.

```
idade >= 18
idade < 18 and possui_autorizacao == True
nota >= 7 and frequencia >= 75
nota >= 7 or frequencia >= 75
not possui_autorizacao
```

Depois teste no Python usando `print()`.

Correção da Atividade 6

Resultados esperados:

```
False
True
False
True
False
```

Observe:

- no `and`, tudo precisa ser verdadeiro;
- no `or`, basta uma condição verdadeira;
- o `not` inverte o resultado.

Essas regras serão usadas na próxima aula para criar decisões com `if`.

Misturando cálculo e comparação

Podemos calcular uma média e depois comparar o resultado.

```
nota1 = float(input("Digite a primeira nota: "))
nota2 = float(input("Digite a segunda nota: "))

media = (nota1 + nota2) / 2

print(f"Média: {media:.2f}")
print(media >= 7)
```

O programa ainda não escreve “aprovado” ou “reprovado”.

Ele apenas mostra se a condição é verdadeira ou falsa.

Atividade 7 — Média e comparação

Crie um arquivo chamado:

`media_comparacao.py`

O programa deve:

1. receber duas notas;
2. calcular a média;
3. mostrar a média formatada;
4. mostrar se a média é maior ou igual a 7.

Teste com:

```
Nota 1: 8
Nota 2: 6
```

A média será `7.00` e a comparação deve retornar `True`.

Erros comuns nesta aula

Alguns erros que podem aparecer:

```
preco = input("Preço: ")
total = preco * 2
```

Nesse caso, `preco` é texto.

Outro erro comum:

```
media = nota1 + nota2 / 2
```

O correto é:

```
media = (nota1 + nota2) / 2
```

Os parênteses mudam o resultado.

Teste de mesa

Antes de executar o programa, podemos testar manualmente.

Exemplo:

```
preco = 40
quantidade = 3
desconto = 10

subtotal = preco * quantidade
valor_desconto = subtotal * desconto / 100
total = subtotal - valor_desconto
```

Tabela:

Variável	Valor
preco	40
quantidade	3

Variável	Valor
`desconto`	10
`subtotal`	120
`valor_desconto`	12
`total`	108

Atividade 8 — Teste de mesa

Faça o teste de mesa no caderno.

```
valor = 200
percentual = 15

acrescimo = valor * percentual / 100
valor_final = valor + acrescimo

print(valor_final)
print(valor_final > 220)
```

Responda:

1. Qual é o valor de `acrescimo`?
2. Qual é o valor de `valor_final`?
3. A comparação retorna `True` ou `False`?

Depois execute no Python para confirmar.

Onde estamos no curso

Até aqui, já trabalhamos:

- lógica e algoritmos;
- entrada, processamento e saída;
- variáveis;
- tipos de dados;
- conversão;
- operadores aritméticos;
- expressões relacionais;
- expressões lógicas;
- estrutura sequencial;
- testes simples de mesa.

Preparação para a próxima aula

Hoje vimos expressões como:

```
media >= 7
idade >= 18
senha == "1234"
produto != ""
```

Na próxima aula, vamos transformar essas condições em decisões:

```
if media >= 7:
    print("Aprovado")
else:
    print("Reprovado")
```

Esse será o início das **estruturas condicionais**.

Desafio final da aula

Crie um programa chamado:

`calculadora_compra.py`

O programa deve receber:

- nome do cliente;
- nome do produto;
- preço unitário;
- quantidade;
- percentual de desconto.

Depois deve mostrar:

- subtotal;
- valor do desconto;
- total a pagar;
- se o total é maior que R$ 100,00.

Ainda não use `if`.

Mostre apenas `True` ou `False` na comparação.

Exemplo de saída esperada

```
Digite o nome do cliente: Maria
Digite o produto: Fone de ouvido
Digite o preço unitário: 80
Digite a quantidade: 2
Digite o desconto: 10

===== RESUMO DA COMPRA =====
Cliente: Maria
Produto: Fone de ouvido
Subtotal: R$ 160.00
Desconto: R$ 16.00
Total: R$ 144.00
Total maior que R$ 100,00? True
```

Fechamento

Para programar bem, não basta decorar comandos.

É preciso entender:

- o que entra no programa;
- quais cálculos precisam ser feitos;
- quais comparações fazem sentido;
- qual resultado precisa ser mostrado.

Hoje usamos cálculos e comparações.

Na próxima aula, vamos usar essas comparações para criar decisões.